Lab4-**1. Local task: Implement quicksort**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Partition function
int partition(int values[], int lo, int hi) {
   int pivot = values[lo];
   int boundary = lo;

   for (int scan = lo + 1; scan < hi; scan++) {
      if (values[scan] < pivot) {
         boundary++;

         int temp = values[boundary];
         values[boundary] = values[scan];
         values[scan] = temp;
      }
   }

   // Place pivot in its final position
   int temp = values[lo];
   values[lo] = values[boundary];
   values[boundary] = temp;

   return boundary;
}
```

```
// Recursive Quick Sort
void quicksort(int values[], int lo, int hi) {
   // Base case: zero or one element
   if (hi - lo <= 1) {
      return;
   }

   int p = partition(values, lo, hi);

   // Sort left and right parts, excluding pivot
   quicksort(values, lo, p);
   quicksort(values, p + 1, hi);
}

// Check if array is sorted
int is_sorted(int values[], int n) {
   for (int i = 1; i < n; i++) {
      if (values[i - 1] > values[i]) {
         return 0;
      }
   }

   return 1;
}

// Comparator used only for testing
int compare_ints(const void *a, const void *b) {
```

```
    int x = *(const int *)a;
    int y = *(const int *)b;

    return (x > y) - (x < y);
}

// Test Quick Sort
void test_sort(const char *name, int original[], int n) {
    int *values = NULL;
    int *expected = NULL;

    if (n > 0) {
        values = malloc(n * sizeof(int));
        expected = malloc(n * sizeof(int));

        if (values == NULL || expected == NULL) {
            printf("Memory allocation failed!\n");
            free(values);
            free(expected);
            return;
        }

        memcpy(values, original, n * sizeof(int));
        memcpy(expected, original, n * sizeof(int));

        // Built-in sorting is used ONLY for testing
        qsort(expected, n, sizeof(int), compare_ints);
    }
```

```
quicksort(values, 0, n);

int correct = is_sorted(values, n);

// Verify that all original values and counts are preserved
if (n > 0 && memcmp(values, expected, n * sizeof(int)) != 0) {
   correct = 0;
}

printf("%-20s : ", name);

if (correct) {
   printf("PASSED\n");
} else {
   printf("FAILED\n");
}

printf("Result: [");

for (int i = 0; i < n; i++) {
   printf("%d", values[i]);

   if (i < n - 1) {
      printf(", ");
   }
}
```

```
    printf("]\n\n");

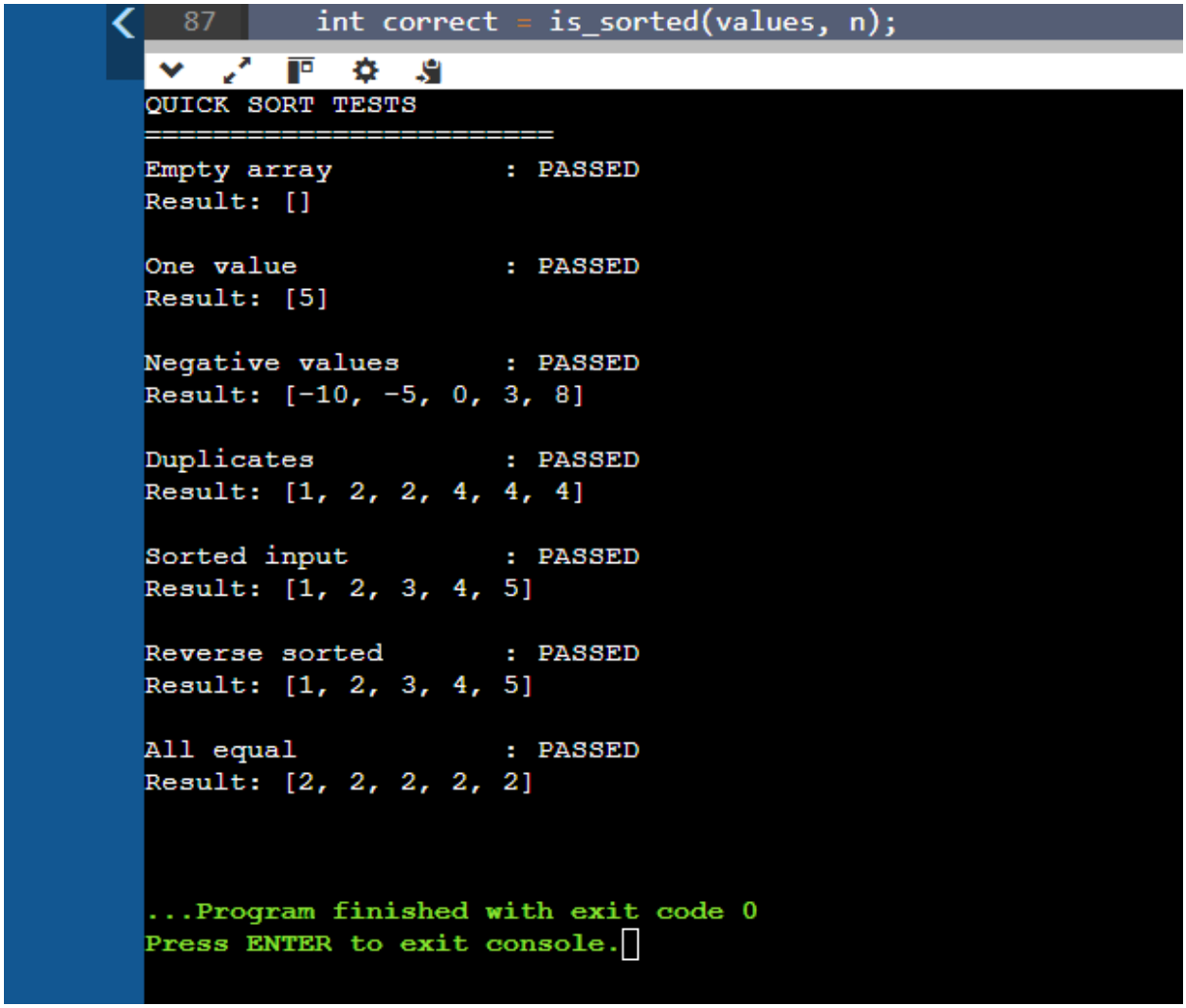
    free(values);
    free(expected);
}

int main() {
    int empty[1] = {0};
    int one[] = {5};
    int negative[] = {-5, 3, -10, 0, 8};
    int duplicates[] = {4, 2, 4, 1, 2, 4};
    int sorted[] = {1, 2, 3, 4, 5};
    int reverse[] = {5, 4, 3, 2, 1};
    int equal[] = {2, 2, 2, 2, 2};

    printf("QUICK SORT TESTS\n");
    printf("========================\n");

    test_sort("Empty array", empty, 0);
    test_sort("One value", one, 1);
    test_sort("Negative values", negative, 5);
    test_sort("Duplicates", duplicates, 6);
    test_sort("Sorted input", sorted, 5);
    test_sort("Reverse sorted", reverse, 5);
    test_sort("All equal", equal, 5);

    return 0;
}
```


87 int correct = is_sorted(values, n);
QUICK SORT TESTS
Empty array : PASSED
Result: []
One value : PASSED
Result: [5]
Negative values : PASSED
Result: [-10, -5, 0, 3, 8]
Duplicates : PASSED
Result: [1, 2, 2, 4, 4, 4]
Sorted input : PASSED
Result: [1, 2, 3, 4, 5]
Reverse sorted : PASSED
Result: [1, 2, 3, 4, 5]
All equal : PASSED
Result: [2, 2, 2, 2, 2]
...Program finished with exit code 0
Press ENTER to exit console.